第8章：线性时间排序

本章内容

- 介绍了几种O(nlgn)的排序算法:
  - 合并排序和堆排序达到此上界;
  - 快速排序平均情况下达到此上界;
- 比较排序算法的下界
- 线性时间排序:
  - 计数排序（Counting Sort）
  - 基数排序（Radix Sort）
  - 桶排序（Bucket Sort）

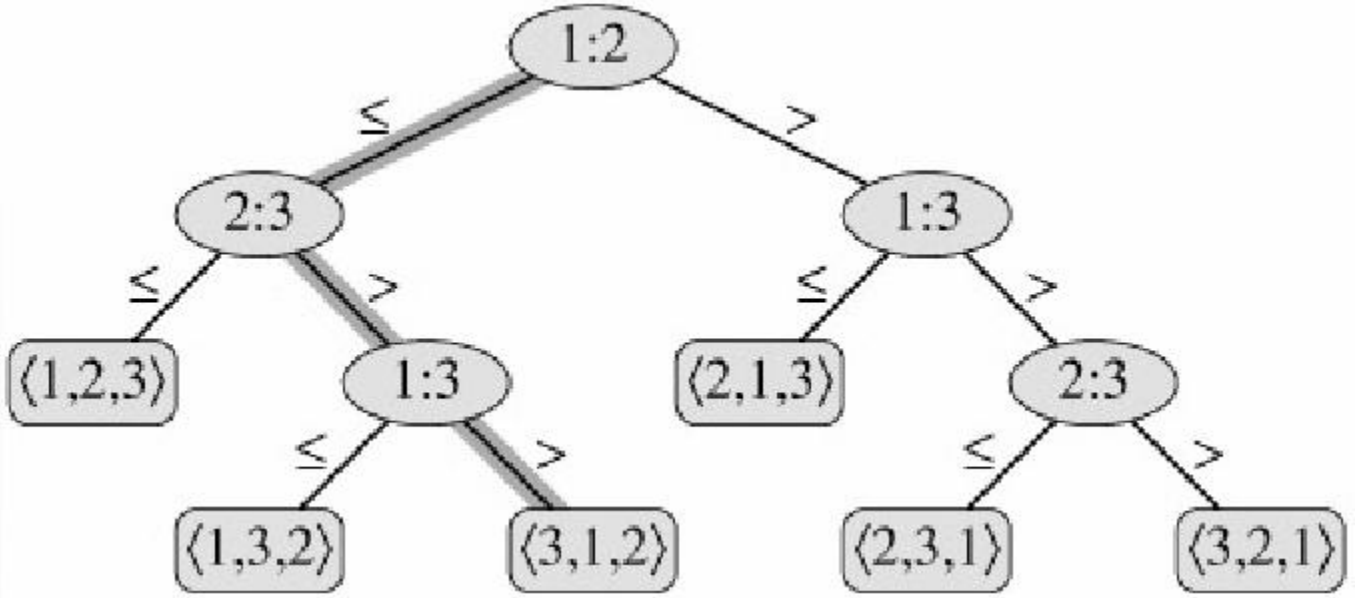
8.1 排序算法的下界

- 比较排序算法的作用
  - 比较排序算法仅用来确定输入序列$<a_1, a_2, \ldots, a_n>$的元素间次序，即给定两个元素a_i和a_j，测试$a_i < a_j$，$a_i \leq a_j$，$a_i = a_j$，$a_i \geq a_j$，或$a_i > a_j$中哪一个成立。

8.1 排序算法的下界

- 决策树模型

  比较排序可以被抽象的视为决策树。一棵决策树是一棵满二叉树，表示某排序算法作用于给定输入所做的所有比较。(6,8,5)



决策树

- 在决策树中，对每个内结点都注明i：j，其中1≤i，j≤n，n是输入序列中的元素个数。
- 对每个叶结点都注明排列$(\pi(1),\pi(2),\ldots,\pi(n))$。
- 排序算法的执行对应于遍历一条从树的根到叶结点的路径。在每个内节结点处要做比较

  $a_i \le a_j$
- 要使排序算法能正确的工作，其必要条件是，n个元素的n!种排列中的每一种都要作为决策树的一个叶子而出现。

最坏情况下界

- 在决策树中，从根到任意一个可达叶节点之间最长路径的长度，表示对应的排序算法中最坏情况下的比较次数。这样，一个比较排序算法中的最坏情况比较次数就与其决策树的高度相等。
- 定理8.1 任意一个比较排序算法在最坏情况下，都需要$\Omega(n\lg n)$次的比较。

定理8.1的证明

- 证明：对于一棵每个排列都作为一个可达叶结点出现的决策树，根据前面的讨论即可确定其高度。考虑一棵高度为h的、具有l个可达叶结点的决策树，它对应于对n个元素所做的比较排序。因为n个输入元素共有n!种排列，每一种都作为一个叶子出现在树中，故有$n! \le l$。
  又由于在一棵高为h的二叉树中，叶子的数目不多于2^h，则有

$$n! \le l \le 2^h$$

  对该式取对数，得到

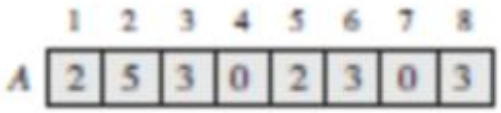
$$h \ge \lg(n!) = \Omega(n \lg n)$$

- 推论8.2堆排序和归并排序是渐进最优的比较算法

证明：堆排序和归并排序的运行时间上界$O(n \lg n)$与定理8.1给出的最坏情况下界$\Omega(n \lg n)$一致

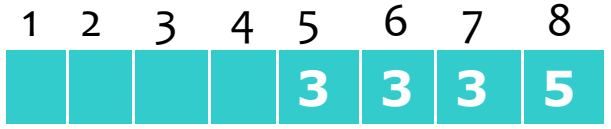
8.2 计数排序

- 计数排序假设n个输入元素中的每一个都是介于0到k之间的整数，此处k为某个整数，当k=O(n)时，技术排序的运行时间是O(n)。
- 计数排序的基本思想就是对每一个输入元素x，确定出小于x的元素个数。有了这一信息，就可以 把x直接放到它在最终输出数组中的位置上。



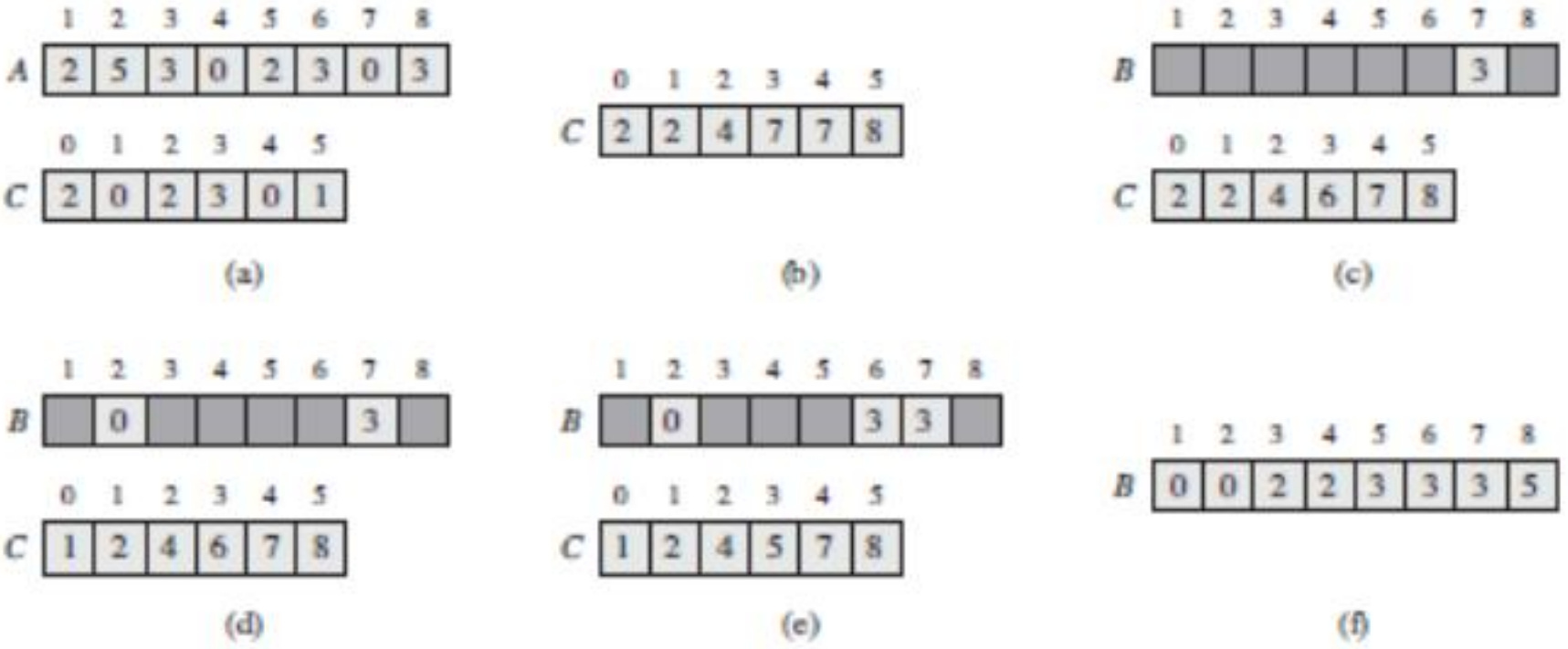
小于等于5的元素个数：8
小于等于3的元素个数：7





计数排序过程

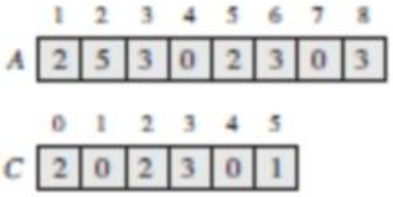
在计数排序算法中，我们假设输入时隔数组A[1..n]，length(A)=n。另外还需要两个数组：存放排序结果的B[1..n]，以及提供临时存数区的C[0..k]

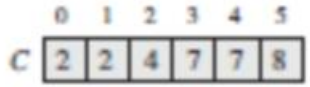


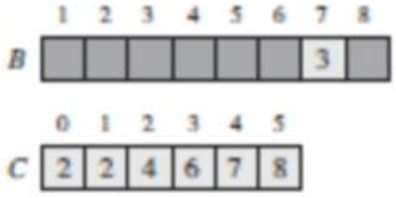
计数排序程序

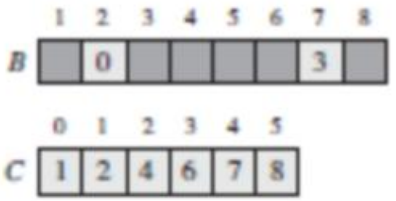
在计数排序算法中，我们假设输入时隔数组A[1..n]，length(A)=n。另外还需要两个数组：存放排序结果的B[1..n]，以及提供临时存数区的C[0..k]

```
COUNT-SORT(A,B,k)
1 for i←0 to k
2  do C[i]←0
3 for j←1 to length[A]
4   do C[A[j]] ← C[A[j]] + 1
5 ▷  C[i] 现在包含等于i的元素个数
6for i← 1 to k
7   do C[i] ← C[i] + C[i - 1]
8 ▷  C[i]现在包含小于或等于i的元素个数
7 for j← length[A] downto 1
8   do B[C[A[j]]] ← A[j]
9       C[A[j]] ← C[A[j]] - 1
```









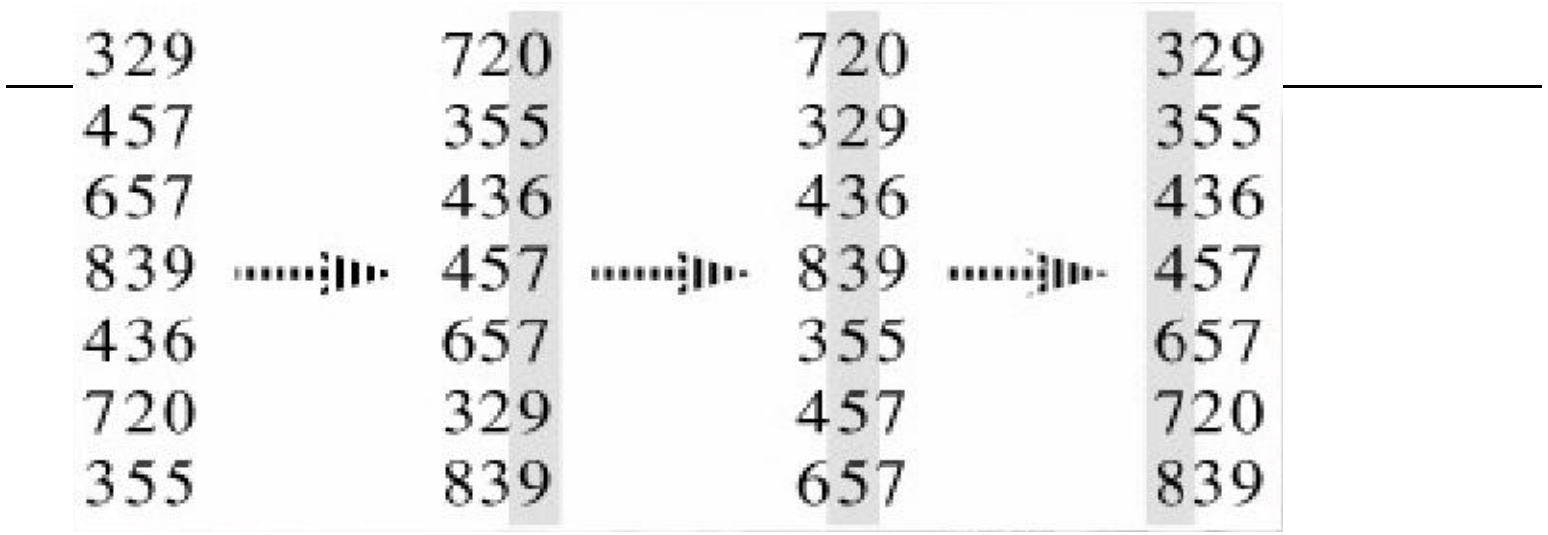
计数排序时间代价

- 计数排序时间代价

  算法第1~2行的for循环所花时间为Θ(k)。第3~4行中for循环所花时间为Θ(n)，第6~7行for循环所花时间为Θ(k)，第9~11行的for循环所花时间为Θ(n)。这样，总的时间就是Θ(k+n)。实践中，当k=O(n)时，运行时间为O(n)。

- 计数排序的特点
  - 计数排序算法没有用到元素间的比较，它利用元素的实际值来确定它们在输出数组中的位置，不是一个基于比较的排序算法，从而它的计算时间下界不再是Ω(nlogn)
  - 由于算法第4行使用了downto语句，经计数排序，输出序列中值相同的元素之间的相对次序与他们在输入序列中的相对次序相同，因此计数排序算法是一个稳定的排序算法。在基数排序中间应用。

8.3 基数排序



- 直觉上，大家可能觉得应该按最重要的一位排序，然后对每个盒子中的数递归地排序，最后把结果合并起来。与人们的直觉相反，基数排序是首先按最低有效位数字进行排序。
- RADIX-SORT(A,d)

  1 for i← 1 to d

  　　do use a stable sort to sort array A on digit i

基数排序定理

- 引理8.3 给定n个d位数，每一个数可以取k中可能的值。如果所用的稳定排序需要$\Theta(n+k)$的时间，基数排序算法能以$\Theta(d(n+k))$的时间正确的对这些数进行排序
- 证明：基数排序的正确性可以通过对正在被排序的列进行归纳而加以证明。对本算法时间代价的分析要取决于选择哪一种稳定的中间排序算法。当每位数字都界于0到k-1之间，且k不太大时，可以选择计数排序。对n个d位数的每一遍处理的时间为$\Theta(n+k)$，共有d遍，故基数排序的总时间为$\Theta(d(n+k))$

基数排序定理

- 引理8.4 给定n个b位数和任何正整数r≤b，RADIX-SORT能在$\Theta((b/r)(n+2^r))$ 时间内正确的对这些数进行排序。
- 证明：对于以一个值r≤b，将每个关键字看做是有 $d=\lceil b/r \rceil$ 个数字，每个数字都包含r位。每一个数字都是介于0到 2^r-1之间的一个整数，这样就可以采用基数排序，此处k= 2^r-1。计数排序的每一遍需要时间为 Θ (n+k)= $\Theta(n+2^r)$ ，共有d遍，总的运行时间为 $\Theta((b/r)(n+2^r))$
- 例如：可以将一个32位的字视为有4个8位的数字，于是有b=32, r=8, k=2^r-1=r^8-1=255, d=b/r=4.

定理说明

- 对于给定的n值和b值，我们希望所选择的r值（r≤b）能够最小化表达式(b/r)(n+2^r)
- 如果b<⌊lg n⌋,则对任何满足r ≤b的r值，都有(n+2^r)=Θ(n)。于是，选择r=b得到的运行时间为(b/b)(n+r^b)=Θ(n),从渐进意义上来看，这一时间是最优的。
- 如果b≥⌊lg n⌋，则选择r=⌊lg n⌋ 可以给出在某一常数因子内的最佳时间，如此选择得到的运行时间为Θ(bn/lgn)。随着将r增大到⌊lg n⌋之上时，分子中的2^r项增加得比分母中的r快，因此，将r增大到⌊lg n⌋之上时得到的时间为Ω(bn/lgn)。如果不是这么做，而是将r减少到⌊lg n⌋之下，则b/r项即会变大，而n+2^r项仍保持为Θ(n)。

基数排序时间

- 基数排序是否要比基于比较的排序算法更好?
  - 如果根据常见的情况有$b=O(\lg n)$并选择$r\approx lgn$，则基数排序的运行时间为$O(n)$，这看上去要比快速排序的平均情况时间$O(nlgn)$更好些。
  - 在这两个时间中隐含在O记号中的常数因子是不同的。对于要处理的n个关键字，尽管基数排序执行的遍数可能要比快速排序要少，但每一遍所需的时间都要长得多。
  - 此外，利用计数排序作为中间稳定排序的基数排序不是原地排序，而很多$O(nlgn)$时间的比较排序算法则可以做到原地排序。因此当内存容量比较宝贵时，像快速排序这样的原地排序算法可能是更为可取的。

8.4 桶排序

- 平均情况下桶排序以线性时间运行
- 假设输入由一个随机过程产生，该过程将元素一致地分布在区间[0，1)上。
- 桶排序的思想就是把区间[0，1)划分成n个相同大小的子区间，或称桶。然后，将n个输入数分布到各个桶中去。因为输入数均匀且独立均匀分布在[0，1)上，所以，一般不会有很多数落在一个桶中的情况。为得到结果，先对各个桶中的数进行排序，然后按次序把各个桶中的元素列出来即可。

8.4 桶排序

- 在桶排序算法的代码中，假设输入是个含n个元素的数组A，且每个元素满足0≤A[i]<1。另外还需要一个辅助数组B[0..n-1]来存放链表实现的桶，并假设可以用某种机制来维护这些表。
- BUCKET-SORT(A)

  1 n←length[A]

  2 for i← 1 to n

  3 　　do insert A[i] into list B[$\lfloor nA[i] \rfloor$]

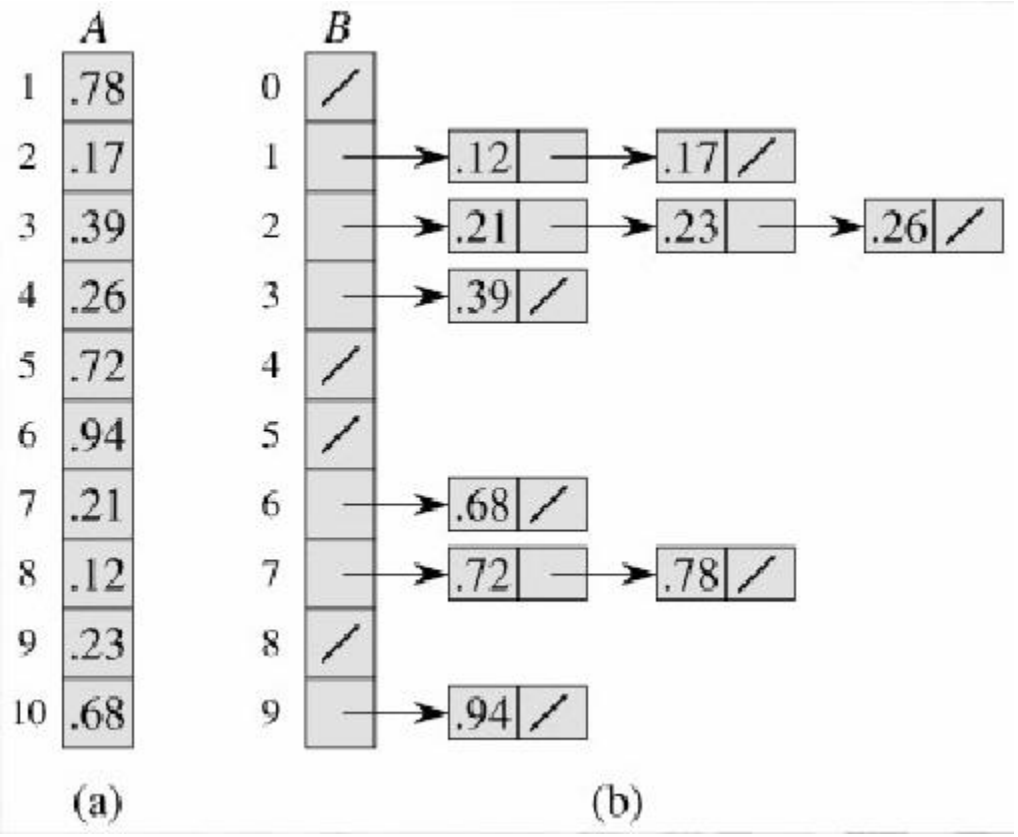
  4 for i← 0 to n-1

  5 　　do sort list B[i] with insertion sort

  6 concatenate the lists B[0]，B[1]，…，B[n-1]together in order

桶排序过程

- 下图演示了桶排序作用于有10个数的输入数组上的操作过程。(a)输入数组A[1..10]。(b)在该算法的第5行后的有序表(桶)数组B[0..9]。桶i中存放了区间[i/10，(i+1)/10]上的值。排序输出由表B[O]、B[1]、...、B[9]的按序并置构成。



桶排序算法的正确性

- 给任意两个元素A[i]和A[j]，不失一般性，假设A[i]≤A[j]。由于floor(nA[i]) ≤floor(nA[j])。，元素A[i]或者被放入A[j]所在桶中，或者被放入一个下表更小的桶中。如果A[i]和A[j]落入同一个桶中，则第4-5行中的for循环会将它们按适当的顺序排列。如果A[i]和A[j]落入了不同的桶中，则第6行将它们按适当的顺序排列，因此，桶排序是能正确工作的。

桶排序算法的运行时间

- 除第5行外，所有各行在最坏情况的时间都是O(n)。唯一需要分析的部分就在于第5行中插入排序所花的时间。
- 为分析调用插入排序的时间代价，设n_i为表示桶B[i]中元素个数的随机变量。因为插入排序以二次时间运行，因而桶排序的运行时间：

$$T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)$$

对上式两边取期望，并利用期望的线性性质，

$$E[T(n)] = E\left[\Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)\right]$$

$$= \Theta(n) + \sum_{i=0}^{n-1} E\left(O(n_i^2)\right)$$

$$= \Theta(n) + \sum_{i=0}^{n-1} O\left(E(n_i^2)\right)$$

桶排序算法的运行时间

- 下式

$$E[n_i^2] = 2 - 1/n$$

对i=0，1，…，n-1是成立的。每一个桶i有相同的值$E[n_i^2]$是不足为奇的，因为输入数组A中的每一个值都是等可能地落在任何桶内的。为了证明上式，我们定义指示器随机变量

$$X_{ij} = I\{A[j]\text{落在桶i中}\}$$

其中，i=0,1,…,n-1, j=1,2,…,n。于是，

$$n_i = \sum_{j=1}^{n} X_{ij}$$

为了计算$E[n_i^2]$，我们将平方项扩展，并对各个项进行重新组合：

桶排序算法的运行时间

$$E\left[n_i^2\right]=E\left[\left(\sum_{j=1}^{n}X_{ij}\right)^2\right]=E\left[\sum_{j=1}^{n}\sum_{k=1}^{n}X_{ij}X_{ik}\right]=E\left[\sum_{j=1}^{n}X_{ij}^2+\sum_{1\le j\le n}\sum_{\substack{1\le k\le n\\ k\ne j}}X_{ij}X_{ik}\right]$$

$$=\sum_{j=1}^{n}E\left[X_{ij}^2\right]+\sum_{1\le j\le n}\sum_{\substack{1\le k\le n\\ k\ne j}}E\left[X_{ij}X_{ik}\right] \qquad (8.3)$$

我们分别计算两个和式。指示器随机变量 X_{ij} 为1的概率为1/n，其他情况下为0.于是有

$$E\left[X_{ij}^2\right]=1*\frac{1}{n}+0*\left(1-\frac{1}{n}\right)=\frac{1}{n}$$

将这两个期望值替换进式8.3，有

$$E\left[n_i^2\right]=\sum_{j=1}^{n}\frac{1}{n}+\sum_{1\le j\le n}\sum_{\substack{1\le k\le n\\ k\ne j}}\frac{1}{n^2}=n*\frac{1}{n}+n(n-1)*\frac{1}{n^2}=1+\frac{n-1}{n}=2-\frac{1}{n}$$

桶排序算法的运行时间

- 因此，桶排序的期望运行时间为$\Theta(n) + n*O(2-1/n)= \Theta(n)$

课堂练习8.1

请给出一个算法，使之对于给定的介于0到k之间的n个整数进行预处理，并能在$O(1)$时间内，回答出输入的整数中有多少个落在区间$[a..b]$内。你给出的算法的预处理时间应是$\Theta(n+k)$。